החוג למערכות מידע
הפקולטה למדעי המחשב והמידע
אוניברסיטת חיפה

Software Engineering and Software Quality Assurance in Information Systems

Assignment 3

Software Development Plan (SDP)

For Ludo Educational Game

Panther Group

Member	Student ID	GitHub username
Islam Hassadya	206858334	IslamHassadya11
Fahid Abu Khadra	325558922	Fahidak22
Rami Aboud	214586208	Ramiaboud2005-dotcom
Ghazal Nahhas	212242937	ghazalnah

Repository Link: https://github.com/IslamHassadya11/SE-Ludo-Panther

Department of Information Systems

University of Haifa

2026-2027

Table of Contents

1. Overview **1**
1.1 Project Summary 1
1.1.1 Purpose, scope and objectives 1
1.1.2 Assumptions and constraints 1
1.1.3 Project deliverables 1
1.1.4 Schedule and budget summary 1
2. References **1**
3. Definitions **2**
5. Managerial Process Plan **2**
5.2 Work Plan 2
5.2.1 Work activities 2
5.2.2 Schedule allocation 4
5.3 Control Plan 5
5.3.1 Requirements control plan 5
5.3.4 Quality control plan for iteration 1 5
5.4 Risk Management Plan 7
6. Technical Process Plans **8**
6.1 Process model 8
6.2 Methods, tools and techniques 9
7. Supporting Process Plans **9**
7.2 Verification and validation plan 9
7.3 Documentation plan 10

1. Overview (Rami, Ghazal)

1.1 Project Summary

1.1.1 Purpose, scope and objectives

This plan describes how the Panther group will develop Ludo, the educational game specified in our SRS sections 1.2 and 2.2 [1]. Following IEEE Std 1058 [3], it covers the work, the schedule, the quality checks and the risks. Our objectives are:

- to deliver a working, runnable version at the end of each of the three iterations;
- to implement every requirement of importance 3 and 2 by the final version;
- to meet the course's criteria: MVC, design patterns, unit tests, tested edge cases, clear documentation and regular GitHub work by every member [2].

The project succeeds when the course staff, our customer, accept each version.

1.1.2 Assumptions and constraints

- Four students, each working about 9 hours a week during an iteration.
- The technology is set by the SRS: Java 21, Java Swing, MVC, a runnable JAR file and the course's template for the game history (SRS sections 2.4 and 3.4).

1.1.3 Project deliverables

Deliverable	Planned date
This plan and the prototype document	01.11.2026
Version 1: runnable JAR file and source code	15.11.2026
Version 2: runnable JAR file, source code and unit tests	06.12.2026
Final version: full system, design document and test plan	20.12.2026
Project closing: closing documents and user guide	03.01.2027

1.1.4 Schedule and budget summary

The planning stage, in which we wrote the SRS, this plan and the prototype, ends on 01.11.2026. Development runs in three iterations: iteration 1 from 02.11 to 15.11, iteration 2 from 16.11 to 06.12, and iteration 3 from 07.12 to 20.12.2026. Each iteration ends with a working version, and the project closing is due on 03.01.2027. The project has no money budget. Its only resource is the team's time. At about 9 hours a week per member, the team has 72 hours per two-week iteration and 108 in iteration 2.

2. References (Ghazal, Rami, Islam, Fahid)

[1] Panther Group. (2026). Software requirements specification for Ludo educational game.

[2] Course staff. (2026). Ludo, version 2 [Project description]. University of Haifa.

[3] IEEE. (1998). IEEE standard for software project management plans (IEEE Std 1058-1998).

[4] Hashem Farhat, N., et al. (2026a). Lecture 2: Pre-project activities [Slides].

[5] Hashem Farhat, N., et al. (2026b). Lecture 3: Reviews [Slides].

[6] Anthropic. (2026). Claude (Opus 5.5 version) [Large language model]. https://claude.ai

3. Definitions (Islam, Fahid)

SRS section 1.3 defines the game and technical terms. This table adds the plan's terms.

Term	Definition	
Acceptance test	The Product Owner's test of whether a story works as the SRS says.	
Defect severity [5]	**1 – Critical**	Prevents an essential function, or endangers security.
	2 – High	Harms an essential function, with no way around it.
	3 – Medium	Harms an essential function, with a way around it.
	4 – Low	Inconveniences users, without harming any function.
	5 – Minor	A cosmetic problem, such as a spelling mistake.
Definition of Done	See section 5.3.4.	
Iteration (sprint)	A two or three week work cycle ending with a runnable version.	
JUnit	A Java tool for writing and running unit tests.	
Maven	A tool that builds the JAR file from the source code.	
Product Owner (PO)	Represents the customer, orders the stories and accepts work.	
Pull request	A request to add code to the main branch after a review.	
Scrum Master (SM)	Keeps the team working by Scrum and removes obstacles.	
UML	Unified Modeling Language: a standard way to draw diagrams of a system's design.	

5. Managerial Process Plan

5.2 Work Plan (Islam, Fahid, Rami, Ghazal)

5.2.1 Work activities

Table 1 shows the three iterations, the goal of each one and its user stories. Iteration 1 builds a playable game on the Easy level, so that the customer can play it and comment early. Iteration 2 extends it to the Medium and Hard levels. The game history and the question management do not affect playing, so they come last, in iteration 3, together with the requirements of importance 1. Until iteration 3, the questions come from the question bank file.

Iteration 1	**Goal: build a playable Easy-level game, from setting up the players to the final ranking.**
US-01	As a player, I want to set the number of players, level, names and colors, so that we can start a game.
US-02	As a player, I want to see the board with its squares and pieces, so that I know where my pieces can go.
US-03	As a player, I want to roll the dice and move a piece by the rules, so that the game moves forward fairly.
US-04	As a player, I want to eat a lone opponent piece and be safe on protected squares, so that the game is competitive.

US-05	As a player, I want to see the final ranking and the game's length when the game ends, so that every player knows his or her final place.
Iteration 2	**Goal: add the Medium and Hard levels with their special squares, complete the remaining rules, and add unit tests for every rule.**
US-06	As a player, I want to play on the Medium and Hard boards with the Boost and UltraBoost dice, so that the game stays challenging as we get better.
US-07	As a player, I want two of my pieces on one square to form a Double Block that opponents cannot pass, so that I can hold back an opponent.
US-08	As a player, I want to answer a course question when I stop on a question square, so that I practice the material.
US-09	As a player, I want surprise squares to trigger random events, so that the game stays exciting.
Iteration 3	**Goal: complete the system with the game history and question management, and finalize the required design patterns and their documentation.**
US-10	As a player, I want every finished game to be saved and listed, so that I see my previous results
US-11	As the admin, I want to log in and manage the questions, so that the questions in the game stay accurate and up to date.
US-12	As a player who is new to Ludo, I want a Rules window, so that I can learn the rules while playing.
US-13	As a player, I want to see the turn order, and the turn to pass when someone stops playing, so that the game never gets stuck.
US-14	As a player, I want to choose English, Hebrew or Arabic, so that I can play in my own language.

Table 1 – Iterations and user stories.

Table 2 breaks the user stories of iteration 1 into tasks, with the SRS requirements of each task, for traceability, and the member who carries it out.

SRS ID	Requirement / User story	Task	Executor
NFR-18, 20, 23	US-01: Setting up a new game	T1.01 Project structure and JAR build setup	All four
FR-01–03		T1.02 Opening screen (View)	Islam
FR-09–11		T1.03 Game setup screen (View)	Islam
FR-10–12, NFR-25		T1.04 Checks of names and colors (Controller)	Fahid
FR-09, 13		T1.05 Game and Player classes (Model)	Fahid
FR-01–03, 09–13		T1.06 Acceptance test of US-01	Ghazal
FR-14, 15		T1.07 Board and its squares (Model)	Rami

SRS ID	Requirement / User story	Task	Executor
FR-16, 17	US-02: Displaying the board	T1.08 Random protection squares (Model)	Ghazal
FR-04, 18, NFR-01–06		T1.09 Game screen with the board (View)	Rami
FR-04, 14–18		T1.10 Acceptance test of US-02	Ghazal
FR-20, 21	US-03: Rolling the dice and moving pieces	T1.11 Rolling the die (Model)	Fahid
FR-22, 24, 26, 32, 39		T1.12 Movement rules (Model)	Islam
FR-19, 27–29		T1.13 Turns and extra rolls (Controller)	Islam
FR-31, NFR-13		T1.14 Selecting and moving a piece (View)	Fahid
FR-19–22, 24, 26–29, 31, 32, 39		T1.15 Acceptance test of US-03	Ghazal
FR-33, 34, 40	US-04: Eating and protecting pieces	T1.16 Eating and protection rules (Model)	Ghazal
FR-33, 34, 40		T1.17 Acceptance test of US-04	Ghazal
FR-41, 54	US-05: Finishing the game	T1.18 End of game and ranking (Model)	Rami
FR-55		T1.19 Game-over screen (View)	Rami
FR-41, 54, 55		T1.20 Acceptance test of US-05	Ghazal

Table 2 – Tasks of iteration 1.

5.2.2 Schedule allocation (Islam, Ghazal)

Iteration 1 lasts two weeks, from Monday 02.11 to Sunday 15.11.2026, and Figure 1 shows when each member works on each task. The project structure (T1.01) is built first, because every other task uses it, and the acceptance test of each story comes after all of its tasks. Iteration 1 has three milestones, each checked by the Product Owner with acceptance tests:

- **M1, Sunday 08.11, setting up a game works.** Players can choose the number of players, the level, their names and their colors, and start a game. The Product Owner has accepted US-01.
- **M2, Thursday 12.11, pieces move on the board.** The Easy board is shown with all of its squares, the die can be rolled, and pieces move by the movement rules. The Product Owner has accepted US-02.
- **M3, Sunday 15.11, a full game can be played to the end.** Pieces can eat and be protected, the game ends with the final ranking, the Product Owner has accepted the remaining user stories, and version 1 is submitted.

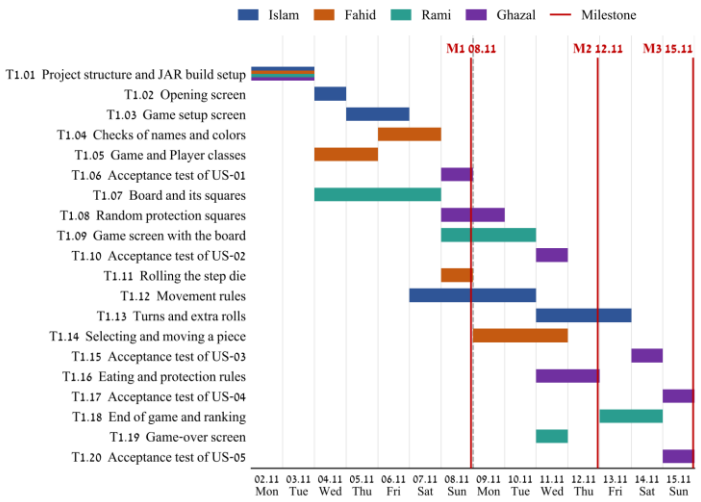


Figure 1 – Schedule of iteration 1 (Monday 02.11 to Sunday 15.11.2026), by member.

5.3 Control Plan (Fahid, Rami)

5.3.1 Requirements control plan

The SRS we submitted in assignment 2 is our requirements baseline. A requirement may change because of the customer, the prototype session or something we learn while building. Every change goes through three steps [4]:

1. **Problem analysis and change specification.** Whoever asks for the change opens a GitHub issue marked "change request" with the requirements involved, the new wording and the reason. The Product Owner checks that it is clear and needed.
2. **Change analysis and costing.** The Product Owner and the developer who owns the affected code use our traceability table (requirement, story, task, test) to find what the change affects, and estimate its hours and its effect on the schedule. Then they decide together with the Scrum Master, after confirming any change to a game rule with the TA.
3. **Change implementation.** An approved change gets a new SRS version, and the backlog, the code and the tests it affects are updated.

While an iteration is running, its scope stays fixed, and approved changes are planned for the next iteration. The only exception is a fix to an importance-3 requirement that fits in the iteration's buffer.

5.3.4 Quality control plan for iteration 1 (Islam, Ghazal, Rami, Fahid)

Every task of iteration 1 must meet the same Definition of Done and is then reviewed at a depth that fits it. A task gets a deeper review when it is more complex, when a failure in it would be more severe, or when one of our risks (section 5.4) affects it [5].

1. Definition of Done. A task counts as done only when all of these are true:

1. Another member approved its pull request before it was merged into the main branch.
2. It works when the game is run from the JAR file, not only inside the development environment (NFR-23).

3. It meets its SRS requirements. Its edge cases were tested by the developer who wrote it, and the reviewer tested them again. Game rules also have passing JUnit tests (NFR-29).
4. It follows the MVC packages (NFR-20) and the Java naming conventions (NFR-22), and its public classes and methods have Javadoc (NFR-37).
5. It has no open defect of severity 1 or 2 (see the severity levels in section 3).
6. If it is the last task of a story, the Product Owner's acceptance test of that story has passed.

2. Review depth. Each task is reviewed at one of four levels and similar tasks are grouped in Table 3. The task we chose not to review in depth is T1.19, the game-over screen, and the last row of the table explains why.

Review	Tasks (see Table 2)	Reason
Code review: a teammate reads the code and runs the JAR file.	T1.02, T1.03, and the acceptance tests T1.06, T1.10, T1.15, T1.17 and T1.20	These screens are simple, and their defects show as soon as the game runs. For each acceptance test, the Scrum Master checks that it covers all the requirements of its story.
Review and testing: a teammate reads the code, runs the JAR file and repeats the developer's test cases.	T1.01, T1.04, T1.05, T1.08, T1.09, T1.11, T1.14, T1.18	These tasks are of importance 3 and contain logic. The build must run on every computer (risk R6), and the game screen must stay fast as the boards grow (risk R5).
Inspection: two teammates check the code with a review checklist [5], and the developer adds JUnit tests for the edge cases.	T1.07, T1.12, T1.13, T1.16	These are the game rules, such as movement (FR-22) and eating (FR-33). They are the most complex tasks, of importance 3, and the base of every later level. Risks R1 (board reuse) and R4 (AI-written code) also concern them.
Reading only: the reviewer reads the code without running it.	T1.19	The screen only shows the ranking and the game's length, which T1.18 (End of game and ranking) computes and which gets a deeper review. It appears once, at the end of a game, so a defect in it is at most an inconvenience. We accept that such a defect may be found only in the acceptance test of US-05.

Table 3 – Review level of each iteration 1 task.

5.4 Risk Management Plan (Islam, Ghazal, Fahid, Rami)

We manage risks in four steps [4]. At every sprint planning, we look for new risks to the project, the product and the business. For each risk, we estimate how likely it is, from very low to very high, and how serious its effect would be: insignificant, tolerable, serious or catastrophic. We then decide how to handle it: we avoid it, we reduce its effect, or we prepare a contingency plan in case it happens. Each member owns two risks and follows them at every sprint planning and retrospective.

Risk	Description, probability and effect, strategy and action
R1 Islam	**The board may not be reusable.** The Medium and Hard levels have larger boards and additional rules. If the board code of iteration 1 fits only the Easy board, the board and the rules that use it would have to be rewritten in iteration 2. The **probability** is high, because nothing in iteration 1 tests the larger boards. The **effect** is serious, because every game rule depends on the board. **Avoidance strategy:** T1.07 (Board and its squares) builds each track from the level's settings instead of fixed numbers, and its inspection also tests the larger tracks.
R2 Islam	**The Medium and Hard levels may take longer than estimated.** The rules of these levels, such as Double Blocks, questions and surprise events, affect each other. Their development may therefore take longer than we estimated, and iteration 2 would be delayed. The **probability** is high, because these are the most complex rules of the game. The **effect** is serious, because iteration 2 also includes the required unit tests. **Contingency plan:** If iteration 2 is behind schedule at its midpoint, its requirements of importance 1 move to iteration 3.
R3 Fahid	**A team member may be unavailable.** Exam periods or illness may keep a team member away at a critical time, and the tasks that depend on him or her would be delayed. The **probability** is high, because iterations 2 and 3 fall in the busiest weeks of the semester. The **effect** is tolerable, because the work is delayed but not lost. **Minimization strategy:** The reviewer of each task also knows it, and if a member is away for more than two days, the Scrum Master moves his or her open tasks to that reviewer.
R4 Fahid	**Code written with AI help may contain mistakes, especially in edge cases.** AI assistants write code that usually compiles and looks correct even when it is wrong, so a mistake in a game rule may go unnoticed. The **probability** is high, because we use these tools for most of our code. The **effect** is serious, because the game rules are of importance 3 and their edge cases are graded [2]. **Avoidance strategy:** Every developer must be able to explain each line of his or her code, and all game-rule code is inspected (section 5.3.4).
R5 Rami	**The game may respond slowly on the larger boards.** The Medium and Hard boards are much larger than the Easy board. On them, the game may lag, so that moves take longer than the required 1 second (NFR-13). The **probability** is moderate, because the

Risk	Description, probability and effect, strategy and action
	problem appears only on the larger boards, but the drawing method is chosen now, in T1.09 (Game screen). The **effect** is serious, because the customer would notice it in every game. **Avoidance strategy:** T1.09 (Game screen) redraws only the pieces that move, and a test board of 188 squares is timed already in iteration 1.
R6 Rami	**The game may not run on the customer's computer.** Computers differ in their environment, such as the Java version or file paths. The game may therefore not start on the customer's computer even though it runs on ours, and a system that cannot be run counts as not submitted. The **probability** is moderate, because these problems are common with default settings. The **effect** is catastrophic, because the grade of the whole round would be lost. **Avoidance strategy:** We use Java 21, UTF-8 and relative file paths, and before each deadline, the JAR file is run on a Windows computer and a macOS computer.
R7 Ghazal	**A requirement may be misunderstood.** Some game rules can be read in more than one way. We may therefore build a rule differently from what the customer meant, and its user story would fail its acceptance test. The **probability** is moderate, because several of our rules are team decisions (TD). The **effect** is serious, because the game rules are of importance 3. **Avoidance strategy:** We ask the TA about unclear rules at each Software Studio session, and every answer that changes a requirement goes through change control (section 5.3.1).
R8 Ghazal	**The customer may prefer different screens.** The customer sees our screens only when a version is shown. The customer may then prefer a different look or behavior, and would not accept the version until the screens are changed. The **probability** is moderate, because the SRS does not describe every detail of the screens. The **effect** is serious, because late changes take time from the next iteration. **Minimization strategy:** We use the comments from the prototype session on 02.11, and show the screens to the TA at each Software Studio session before submitting.

Table 4 – Risks.

6. Technical Process Plans (Ghazal, Rami)

6.1 Process model

We work with Scrum, in three iterations with the dates given in section 1.1.4. Before iteration 1, we wrote the SRS, this plan and the prototype. Every iteration follows the same cycle:

Event	When	What happens
Sprint planning	First day	The Product Owner presents the stories in order of importance, and the team splits them into tasks, estimates them and commits.
Daily scrum	Every day, in the group chat	Each member writes what he or she did, what comes next and what is in the way.

Event	When	What happens
Development	Throughout	Each task is built on its own branch, reviewed and merged.
Sprint review	Last day	We run the version from its JAR file and check it against the iteration goal.
Retrospective	After the review	We discuss what to improve, and our risks.

Each version is marked in GitHub (v1.0, v2.0, v3.0). After iteration 3 we submit the final version and hold a last retrospective on the whole project. All four of us develop in every iteration, and two members also take a second role, which changes every iteration:

Iteration	Scrum Master	Product Owner	Developers only
1	Fahid	Ghazal	Islam, Rami
2	Rami	Islam	Fahid, Ghazal
3	Islam	Fahid	Rami, Ghazal

6.2 Methods, tools and techniques

- **Language and screens:** Java 21 and Java Swing. Maven builds the JAR file the same way on every computer.
- **Architecture:** MVC in separate packages. The Model updates the screens through the Observer pattern, and we also use Singleton, Factory and Strategy.
- **Version control:** Git, with our shared repository on GitHub. Every task has its own branch and is merged only through a reviewed pull request. GitHub Projects is our task board, and GitHub Issues hold defects and change requests.
- **AI assistants**, such as Claude [6], help write code and revise the grammar of our documents.
- **Testing and documentation:** JUnit 5, Javadoc, UML diagrams and the Java naming conventions.

7. Supporting Process Plans (Islam, Fahid)

7.2 Verification and validation plan

This plan covers verification and validation for the whole project, and section 5.3.4 adds the details for iteration 1. Every piece of work is checked by someone other than the person who made it. Table 5 lists the techniques, their tools and their owners [3].

Technique	How and when	Tool	Responsible
Verification			
Traceability	Each task and test is linked to its SRS requirement, and the links are checked at each sprint review.	GitHub Issues	Product Owner

Technique	How and when	Tool	Responsible
Peer reviews	Every pull request, at the level set in section 5.3.4.	Pull requests	Another developer
Progress reviews	The daily scrum, and each milestone.	GitHub Projects	Scrum Master
Milestone reviews	Each sprint review, and a review of the UML design in iteration 2.	JAR file, UML	Scrum Master
Prototyping	Other teams try our prototype on 02.11.	Prototype	Fahid
Validation			
Testing	Unit tests of every game rule before each version (NFR-29), and full games in iteration 3.	JUnit 5, Maven	Each developer
Acceptance tests	Each story is tested against its SRS requirements.	Test lists	Product Owner
Demonstration	Each version is run from its JAR file for the customer.	JAR file	Whole team
Analysis	A statistical test of the dice (NFR-28), and timing of response times.	JUnit 5	Developers
Inspection	Game-rule code is checked with a review checklist.	Checklist	Two reviewers

Table 5 – Verification and validation techniques.

7.3 Documentation plan

Document	Standard	Written by	Reviewed by	Due	Distribution
SRS	IEEE 830	All members	PO, Course staff	Done	Staff, team
SDP	IEEE 1058	All members	All members	01.11.2026	Staff, team
Prototype document	Course template	All members	PO	01.11.2026	Staff, team
Design document	UML	Developers	SM of the iteration	Iterations 2 and 3	Staff, team
Test plan (STP)	IEEE 829	Developers	PO of iteration 3	20.12.2026	Staff, team
User guide	README	PO of iteration 3	All members	03.01.2027	Staff, users

Table 6 – Documents are all kept in GitHub.